

LAB ASSIGNMENT

Cooperative Multi-Agent Gridworld

Design, implement, and evaluate two autonomous agents that must coordinate in a shared environment.

LAB	2	POINTS	100
ASSIGNED	8/31/2026	DUE	9/4/2026

OBJECTIVE

Design, implement, and evaluate two autonomous agents that operate in a shared gridworld. Compare an independent baseline with a coordinated policy using PEAS, task-environment classification, event logs, tests, and experimental evidence.

Learning Objectives

- Describe a multi-agent task environment using PEAS and task-environment properties.
- Implement distinct agent programs that map local percepts and internal state to actions.
- Model partial observability, simultaneous actions, collision resolution, and shared resources.
- Use structured messages, claims, and a deterministic rule to coordinate cooperative behavior.
- Compare policies with a defined performance measure and reproducible experimental evidence.

Required Resources

- A programming environment suitable for implementing a small discrete-event simulator.
- A way to save source code, tests, map configurations or seeds, results, and event logs.
- Artificial Intelligence: A Modern Approach and the course materials on intelligent agents.

COURSE READING ALIGNMENT

Artificial Intelligence: A Modern Approach (Russell and Norvig): intelligent agents, PEAS, rationality, task-environment properties, agent architectures, and multi-agent interaction.

PART 1

System Design at a Glance

Build a shared gridworld, then compare independent and coordinated multi-agent behavior.

Required Components

PART	FOCUS	IMPLEMENT	EVIDENCE
1	Environment	8 x 8 grid, local sensing, simultaneous joint actions	Solvable map and action-resolution rules
2	Baseline	Independent policies; no messages or claims	Baseline trace and score
3	Coordination	Messages, memory, claims, deterministic conflict rule	Message-to-action evidence
4	Evaluation	Same three fixed maps or seeds; 60-step horizon	Results table, tests, event logs

Required Workflow

1 DESIGN WITH PEAS Write PEAS and classify observability, dynamics, sequence, discreteness, and agents.	2 BUILD THE SIMULATOR Implement state, legal actions, collision resolution, packages, score, and event log.
3 IMPLEMENT THE BASELINE Run independent agents without messages or claims and save a trace.	4 ADD COORDINATION Implement structured messages, internal state, claims, and a tie-break rule.
5 TEST AND MEASURE Run both policies on the same three maps or seeds; save results and tests.	6 ANALYZE THE EVIDENCE Explain how coordination changed behavior using results and log entries.

Coordination Protocol

[DISCOVER] announce a visible package	[CLAIM] announce the package being targeted	[RELEASE] announce delivery or an abandoned goal
LOCAL MEMORY known packages, current goal, active claims	TIE-BREAK RULE document a deterministic conflict rule	LOG THE MESSAGE time, sender, message, later action

SCENARIO

Cooperative Supply Delivery

Two robots must collect packages and deliver them to a shared base while operating with incomplete local information.

BUILD

Environment and Agent Interface

PERCEPTS • ACTIONS • JOINT ACTION RESOLUTION

Purpose: Build a shared gridworld that separates environment state from each agent program.

IMPLEMENTATION BOUNDARY

Keep each agent's local percept distinct from environment state. The environment applies simultaneous joint actions and produces the next percepts.

REQUIRED CONFIGURATION

- 8 x 8 grid: one base, at least five obstacles, at least four packages, and one intentional bottleneck.
- A local 3 x 3 percept; carried item, last-action status, clock, and incoming messages.
- Actions: move N/S/E/W, wait, pick up, drop, and one short structured message.
- Document observe -> act -> environment.step -> run_episode; block and log same-square and swap attempts.

BASELINE

Independent Baseline

LOCAL PERCEPTS • MEMORY • NO COORDINATION

Purpose: Establish a fair reference policy: agents act separately, use only their own percepts and memory, and ignore messages and claims.

BASELINE RULE

Document the local decision rule before comparing it with coordination. The baseline must use the same maps and scoring function as the coordinated agents.

REQUIRED BEHAVIOR

- Implement a sensible local rule that explores, collects, and delivers packages.
- Each agent may keep local memory, but neither receives or acts on coordination messages.
- Run a baseline episode and save an event log showing action choices and outcomes.
- Describe what each agent can know and what remains hidden because sensing is local.

LOCAL CONTROL, SHARED ENVIRONMENT

The simulator owns the grid state and resolves joint actions. It must not choose actions for the agents or use a central controller to coordinate them.

COORDINATION

Messaging and Evaluation

Implement a protocol, then use comparable experiments to evaluate its effects.

BUILD

Coordinated Policy

MESSAGES • INTERNAL STATE • CONFLICT RESOLUTION

Purpose: Extend the agents so they share discoveries, allocate packages, and avoid interfering with each other.

MESSAGE FORMAT

A message must contain only a short structured payload, such as DISCOVER(package, location), CLAIM(package, agent), or RELEASE(package).

REQUIRED PROTOCOL

- Use DISCOVER(package, location), CLAIM(package, agent), and RELEASE(package).
- Maintain known packages, current goal, and active claims in each agent's internal state.
- State and use a deterministic rule for conflicting claims or simultaneous goals.
- Show a message that changes a later action or prevents an avoidable conflict.

TEST

Experiments and Tests

SAME MAPS • SCORE • REPRODUCIBLE EVIDENCE

Purpose: Compare independent and coordinated behavior using the same scenarios and an explicit team performance measure.

RESULTS RECORD

For every run, retain the map or seed, policy, deliveries, team score, collision attempts, and steps to completion or horizon.

REQUIRED EVIDENCE

- Run both policies on the same three fixed maps or seeds; stop at completion or 60 steps.
- Record deliveries, score, collision attempts, and steps to finish or horizon.
- Test collision resolution, exactly-once delivery, message timing, and claim behavior.
- Team score: +10 per delivery; -1 per move; -2 invalid action; -3 collision; -5 undelivered.

MESSAGE TIMING MATTERS

A message sent at time t must be available before the recipient's decision at the next time step. Log the send, receive, and downstream action.

Evidence and Analysis

Record reproducible runs and cite event-log evidence for claims about coordination.

Required Documentation

☐ Map configuration or seed and horizon	☐ Baseline or coordinated policy
☐ Initial base, obstacles, packages, and agent locations	☐ Score: deliveries, moves, invalid actions, collisions, undelivered packages
☐ Event log: percept summary, message, action, and result	☐ Test outcomes and documented resolution order
☐ Comparison results from the same three maps or seeds	☐ Specific time steps cited in the analysis

Event Log and Results Format

RUN METADATA	<i>[map/seed] [baseline or coordinated] [horizon]</i>
INITIAL STATE	<i>[base, obstacles, packages, and starting locations]</i>
EVENT LOG	<i>[t agent percept summary received/sent message action result]</i>
RESULTS TABLE	<i>[run policy deliveries score collision attempts steps]</i>
INTERPRETATION	<i>[what the log shows about coordination or interference]</i>

Minimum Test Cases

[COLLISION]	Same-destination or swap attempt: both agents are blocked and the result is logged.
[DELIVERY]	Pickup, carry, and delivery update shared state exactly once.
[MESSAGE]	A message sent at time t becomes available before the next decision.
	FAIR COMPARISON Run both policies unchanged in the same map configurations. A difference in results should be attributable to coordination rather than a different environment.

Comparative Analysis Report

Use PEAS, task-environment properties, results, and log evidence to explain coordination.

LENGTH AND EVIDENCE

Write 750-1000 words. Support claims with your PEAS description, result table, and specific event-log time steps. Distinguish observed behavior from assumptions about agent reasoning.

PEAS and Environment Classification

LENS	WHAT TO DOCUMENT	WHY IT MATTERS
PEAS	Performance, Environment, Actuators, Sensors Show how the task is specified and how each agent maps percepts to actions.	Agent Design Explain the information, actions, and objective available to each autonomous agent.
TASK ENVIRONMENT	Discrete, sequential, partially observable, dynamic, cooperative multi-agent Justify each property from your simulator, including the effect of the other agent.	Design Consequences Connect the classification to memory, messaging, collision handling, and coordination choices.

Recommended Report Structure

1. PEAS and classification Explain the task environment and justify each property.	2. Independent baseline Describe its percepts, local memory, rule, and limitations.
3. Coordinated policy Explain messages, claims, shared information, and tie-break.	4. Results Compare performance across the three maps or seeds.
5. Causal evidence Cite a message and later action that changed allocation or interference.	6. Limitations / extension Analyze a remaining failure, edge case, tradeoff, or alternative architecture.

Analytical Questions

What did each agent know, and what was missing because sensing was local?	How did a message or claim allocate work or prevent interference?
Which score terms explain the performance difference?	What failure or edge case remains, and why?
How would the design change in a competitive setting?	CITE SPECIFIC EVENT-LOG EVIDENCE

SUBMIT

Deliverables, Evaluation Criteria, and Final Checklist

Submit the required materials according to the instructor's submission instructions.

Deliverables

A	Source Code	Environment, agent policies, and automated tests.
B	README and Evidence	Run instructions, PEAS, classification, policy descriptions, results table, and selected event logs.
ANALYSIS		750-1000-word comparison of the baseline and coordinated policies with specific evidence.

Evaluation Criteria

COMPONENT	EVIDENCE EXPECTED	REVIEW
Task environment and interface	Correct PEAS, legal actions, percepts, state, and resolution order.	Complete
Independent baseline	Separate agent policy and reproducible baseline trace.	Complete
Coordinated policy	Protocol, memory, claims, tie-break, and observed impact.	Complete
Testing and experiments	Required tests and comparable results from three maps or seeds.	Complete
Analysis	Evidence-based interpretation of outcomes, limitations, and extension.	Complete

Final Submission Checklist

☐ Environment includes the required grid, base, obstacles, packages, and bottleneck.	☐ Independent and coordinated policies use the same environment and maps or seeds.
☐ Collision, delivery, message-timing, and claim tests pass.	☐ Event logs include time, agent, percept, message, action, and result.
☐ Results table reports deliveries, score, collision attempts, and steps.	☐ PEAS and task-environment classification are in the README or report.
☐ Analysis cites specific log entries and compares both policies.	☐ README explains how to run the simulator, tests, and experiments.